

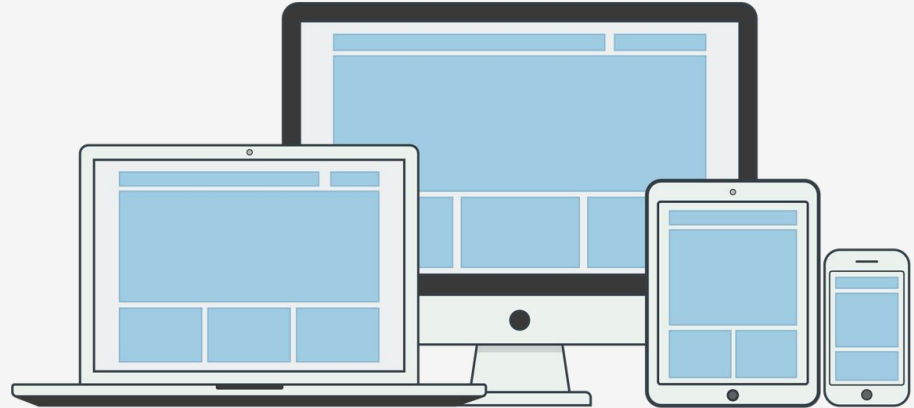
MULTI-CORE, PARTITION-BASED SCHEDULING IN REAL TIME OPERATING SYSTEM

Ikramul Hassan Sazid

Hamm-Lippstadt University of Applied Sciences

Real-Time Systems – SS26

Prof. Dr. Stefan Henkler



INTRODUCTION: THE MULTI-CORE SHIFT

Modern embedded systems increasingly require more and more processing power for concurrent features in modern technology, for example, in automotive and avionics.

- Processing units have to execute multiple task threads,
- for Hard Real-Time (HRT) systems, missing a deadline renders the task as failed,
- Parallel computing and scheduling becomes a necessity to handle heavy concurrent tasks.

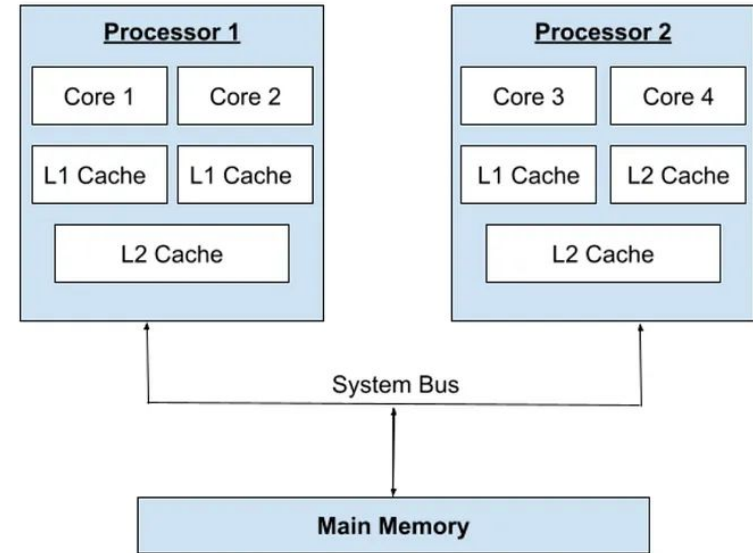


Fig. 2: Multi-core Processor Architecture

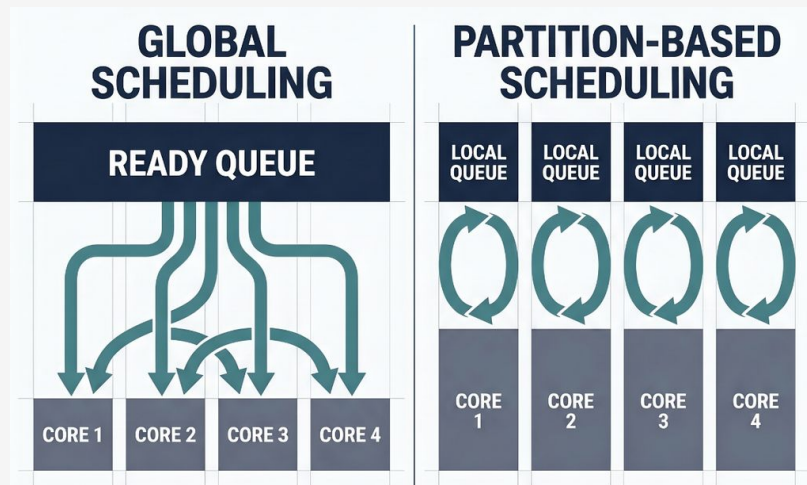
GLOBAL VS. PARTITIONED REAL-TIME SCHEDULING

Global Scheduling:

- one global queue, any core
- Free migration, task can be preempted and resume on different core
- Global scheduling maximizes processor utilization

Partitioned Scheduling:

- Dedicated queue for every single processing core, each running their own scheduler
- Static, tasks is bound to one core and cross-core migration is forbidden
- partitioning eliminates migration overhead and saves valuable processing power

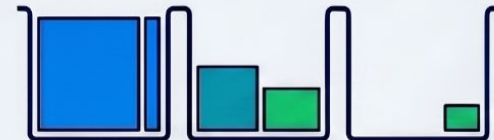


PARTITIONED REAL-TIME SCHEDULING: 4 MAIN APPROACHES

- **Strict P-EDF:** tasks permanently assigned to individual processors before to execution by bin-packing methods like First-Fit Decreasing (FFD) or Best-Fit Decreasing (BFD)
 - *First-Fit Decreasing (FFD):* Task are sorted by utilization to the first core with enough remaining capacity
 - *Best-Fit:* Minimizes wasted space on a single core
 - *Worst-Fit:* Spreads the load across all available cores
- **Semi-Partitioned /Task-Splitting Scheduling:** Hybrid approach where at most $m-1$ tasks are split between two cores to optimize system utilization.
- **Clustered Scheduling (C-EDF):**

First-Fit Decreasing (FFD)

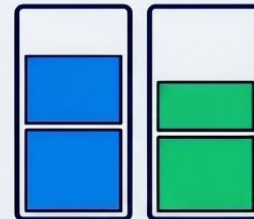
TASKS (Sorted by Utilization)



PROCESSOR 1 PROCESSOR 2 PROCESSOR 3

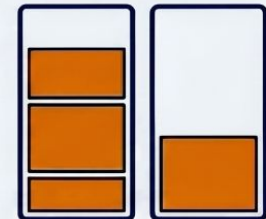
Best-Fit vs. Worst-Fit

BEST-FIT



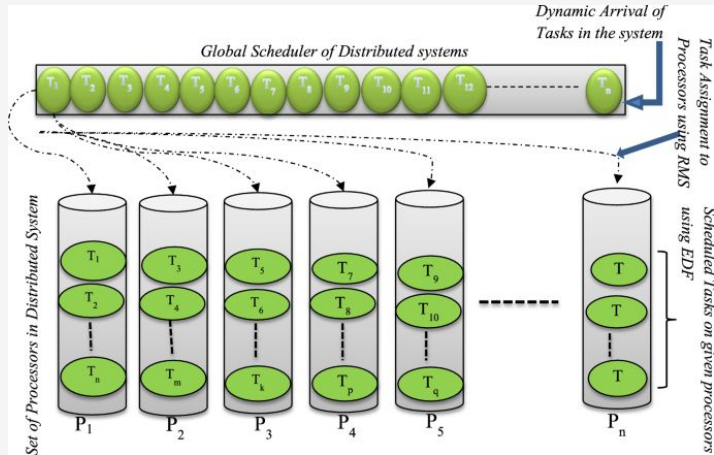
PROCESSOR A PROCESSOR B

WORST-FIT



PROCESSOR C PROCESSOR D

PARTITIONED EDF (P-EDF)



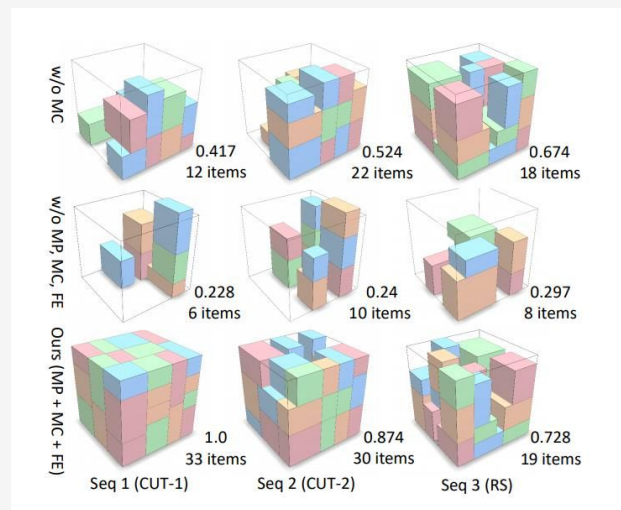
[2]

- Offline Phase: Tasks are statically mapped to specific cores. This is a Bin-Packing problem (NP-Hard).
- Runtime Phase: Each core executes the standard Earliest-Deadline-First (EDF) algorithm independently.
- Zero Migration: Eliminates cache misses and bus contention caused by task migration.
- Predictability: Allows for rigorous uniprocessor schedulability analysis on each core.
- Heavy Task: Struggles with heavy task as it cannot be divided among other cores and can take big chunks of cpu time.

THE FIRST-FIT DECREASING (FFD) HEURISTIC

Since bin-packing is complex and mathematically perfect task distribution is impossible, heuristics like FFD are used.

- FFD provides a fast, polynomial-time approximation to effectively organize the workload
- FFD is strongly preferred for P-EDF because it guarantees a low "speedup factor" (bounded at $(4/3) - (1/3m)$)
- "Heaviest" tasks first, avoids dilemma if the heaviest are left until the end for not having enough cpu space free in any cores



Overhead-aware EDF schedulability: why G-EDF fails in hard real-time

Real RTOS overhead inflates each task's WCET. Migration-driven global scheduling inflates it far more, collapsing the schedulable utilization ceiling.

THE SCHEDULABILITY TEST

$$U_{\text{nom}} + U_{\text{OH}} \leq m$$

nominal utilization + overhead utilization must fit on m cores

$$U_{\text{OH}} = \sum (\Phi_i / T_i)$$

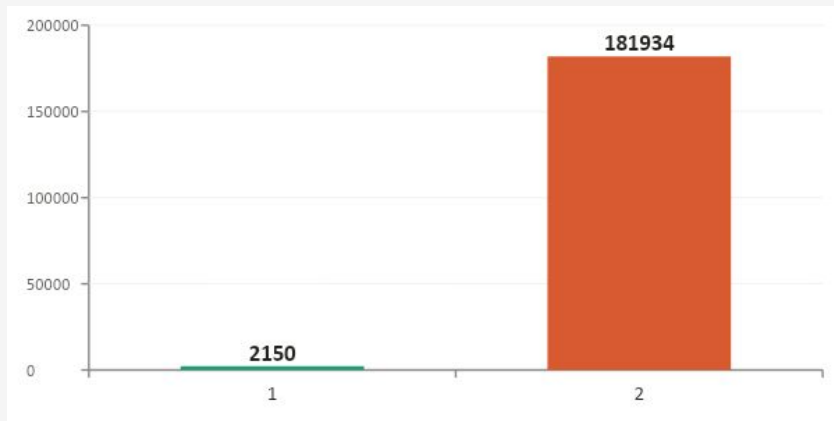
overhead utilization summed over all tasks

$$\Phi = \delta_{\text{sched}} + \delta_{\text{ctxs}}$$

per-job worst-case overhead = scheduling + context-switch cost

Schedulable ceiling: $U_{\text{max}} = m - U_{\text{OH}}$

Scheduling overhead per job — 99th percentile (CPU cycles)



P-EDF • PER-JOB Φ

3,140 cycles

2,150 sched + 990 ctx-switch

G-EDF • PER-JOB Φ

184,518 cycles

181,934 sched + 2,584 ctx-switch

OVERHEAD PENALTY

$\approx 59\times$ worse

G-EDF inflates every WCET $\sim 59\times$ more

PRACTICAL REALITIES: RTOS OVERHEADS



Context Switching

Latencies for saving/restoring task states.



Scheduling

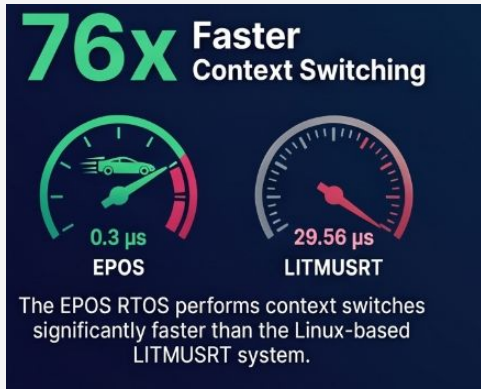
CPU time spent calculating the next priority task.



CPMD Delay

Cache loss when a task is preempted.

PERFORMANCE: P-EDF VS. G-EDF



RTOS vs Linux patches in context switching

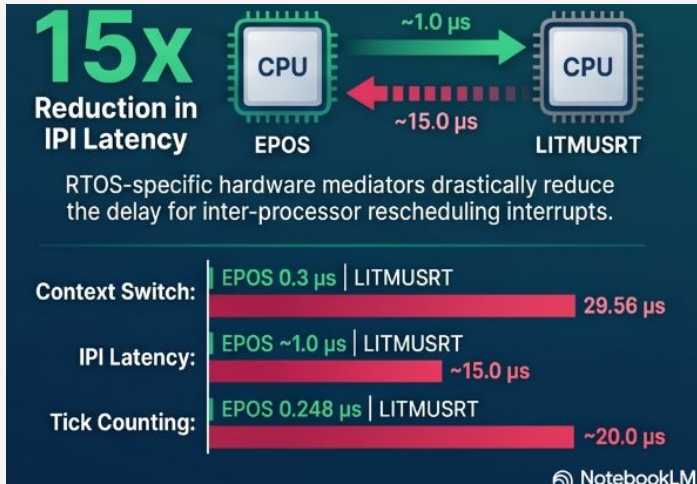
Overhead

Since G-EDF allows task migration, it introduces scheduling overhead. 'Context switching' can cause latency and consume processing power.

Cache-Related Preemption and Migration Delay (CPMD) Delay

When a task is preempted, "cache" memory is overwritten and the new core has to fetch the instruction again, causing performance hit.

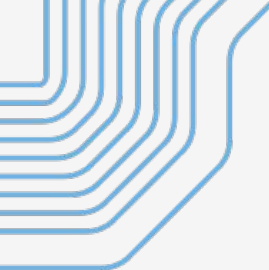




Hard Real-Time Environments

Partitioned scheduling outperforms Global in Hard Real-Time environments because it avoids migration-related cache invalidation and global lock contention.

P-EDF consistently achieves higher schedulability ratios than G-EDF across most task distributions



Limitations & Future Work

Sub-optimality

Bin-packing might leave some cores underutilized while others are full.

Shared Resources

Inter-core contention for shared L3 cache and memory buses still exists.

Semi-Partitioning

Exploring task splitting to allow partial migration for better balance.



References:

- [1] R. Joshi, "Multi-Core Processor Architecture," *Medium*, 2020. [Online]. Available: <https://riya-joshi19.medium.com/multi-core-processor-architecture-f37fba7a1c84>. [Accessed: Jun. 22, 2026].
- [2] R. Sharma, N. Nitin, M. A. R. AlShehri, and D. Dahiya, "Priority-based joint EDF–RM scheduling algorithm for individual real-time task on distributed systems," *The Journal of Supercomputing*, vol. 77, no. 1, pp. 890-908, Jan. 2021, Fig. 7. [Online]. Available: <https://link.springer.com/article/10.1007/s11227-020-03306-x#Fig7>. [Accessed: Jun. 22, 2026].
- [3] "Python example of 3D bin packing problem and visualization," *Stack Overflow*, Aug. 2021. [Online]. Available: <https://stackoverflow.com/questions/68953770/python-example-of-3d-bin-packing-problem-and-visualization>. [Accessed: Jun. 22, 2026].
- [4] G. Gracioli, A. A. Fröhlich, R. Pellizzoni, and S. Fischmeister, "Implementation and evaluation of global and partitioned scheduling in a real-time OS," *Real-Time Systems*, vol. 49, no. 6, pp. 699-752, Nov. 2013.
- [5] S. Baruah, "Partitioned EDF scheduling: a closer look," *Real-Time Systems*, vol. 49, pp. 675-698, Nov. 2013.
- [6] P. Han, W. Hu, Z. Zhai, and M. Huang, "A Model-Based Optimization Method of ARINC 653 Multicore Partition Scheduling," *Aerospace*, vol. 11, no. 2, 2024.
- [7] A. Bastoni, B. B. Brandenburg, and J. H. Anderson, "An Empirical Comparison of Global, Partitioned, and Clustered Multiprocessor EDF Schedulers," in *Proc. 31st IEEE Real-Time Systems Symposium*, 2010, pp. 14-24.